



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Performance Analysis of Incremental Learning in Network Intrusion Detection

Student: Nguyễn Quang Minh 20194444
Supervisor: Dr. Trần Hải Anh

ONE LOVE. ONE FUTURE.



CONTENT

1. INTRODUCTION
2. PRELIMINARIES AND LITERATURE REVIEW
3. METHODOLOGY
4. NUMERICAL RESULTS
5. CONCLUSION





HUST

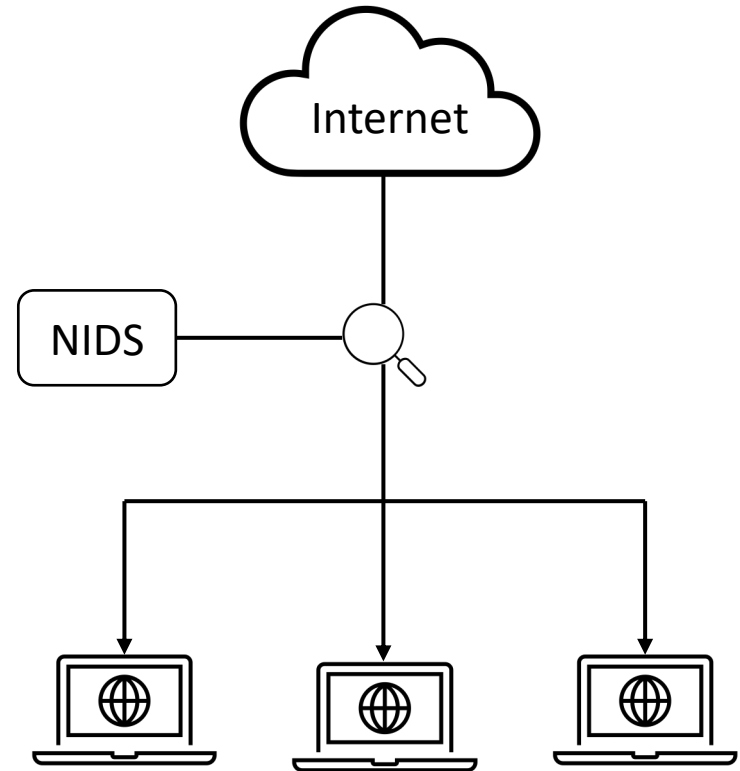
1. INTRODUCTION

Introduction

Network Intrusion Detection System (NIDS) monitors network traffic by **analyzing data packets** at network point to identify malicious actions.

Supervised neural network involves training a model on labeled dataset where the input data is associated with known labels.

When new attacks emerge, the model failed to classify. =>
Incorporating Class Incremental Learning (CIL) to detect new attack types.



Class Incremental Learning

- Stream of incoming new classes.
- No overlapping classes.
- Training data limited to current task.

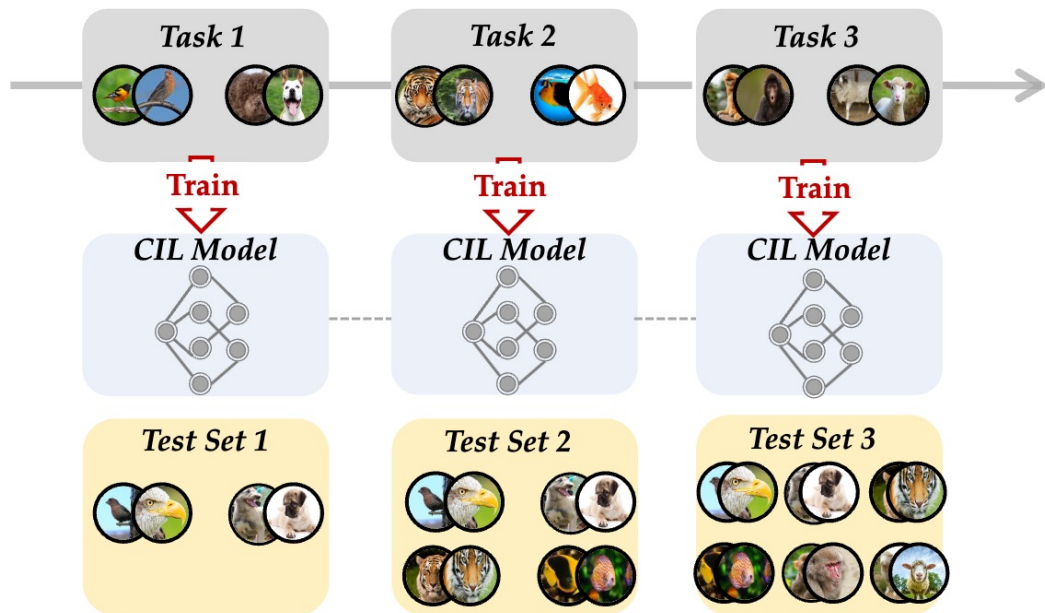


Fig 1: Setting of CIL.

Source: Deep Class Incremental Learning: A survey

Benefits:

- Adaptability -> Allow NIDS to detect new attack classes
- Compute Resource Efficiency -> Reduce deployment time of NIDS
- Memory Efficient.

Challenges: Catastrophic forgetting

Catastrophic forgetting refers to the tendency of a neural network to completely previously learned information upon learning new data.

- Integration of Class Incremental Learning in Network Intrusion Detection System
- Comparative Analysis of CIL methods:
 - LWF: Learning Without Forgetting
 - ICARL: Incremental Classifier and Representation Learning
 - DER: Dynamic Expandable Representation
 - MEMO: Memory Efficient Expandable Model
- Evaluations on multiple NIDS datasets: KDD99, CIC-IDS-2017, UNSW-NB15, ToN-IoT-Network and a synthetic dataset.



2. PRELIMINARIES AND LITERATURE REVIEW



Preliminaries

Feature extractor $\Phi(x)$: Transform raw input into more useful information.

Fully Connected (FC) layer $\mathcal{F}$: Perform classification on the extracted features.

Loss function $\mathcal{L}$: Measures how well the model predict.

Prediction: $\hat{y} = \operatorname{argmax}(\mathcal{F}(\Phi(x)))$

Cross Entropy Loss: $\mathcal{L} = -y * \log(\mathcal{S}(\mathcal{F}(\Phi(x))))$

Expanding FC layer

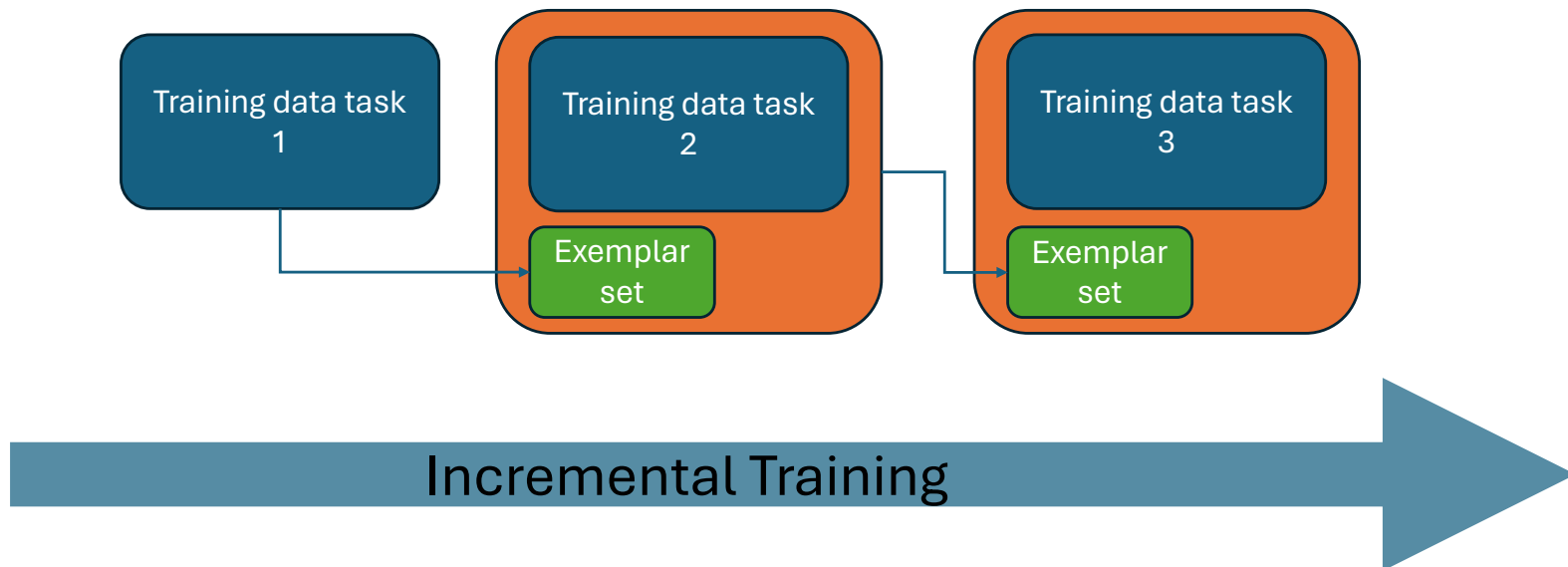
The output layer match the number of classes, so $\mathcal{F}$ must be expanded whenever facing new task.

$$\mathcal{F}_t = \mathcal{F}_{t-1} + \mathcal{F}'_t$$

Preliminaries

- **Exemplar set** is an extra collection of instances from former tasks

$$\varepsilon = \{x_j, y_j\}_{j=1}^M, y_j \in Y_{b-1}.$$

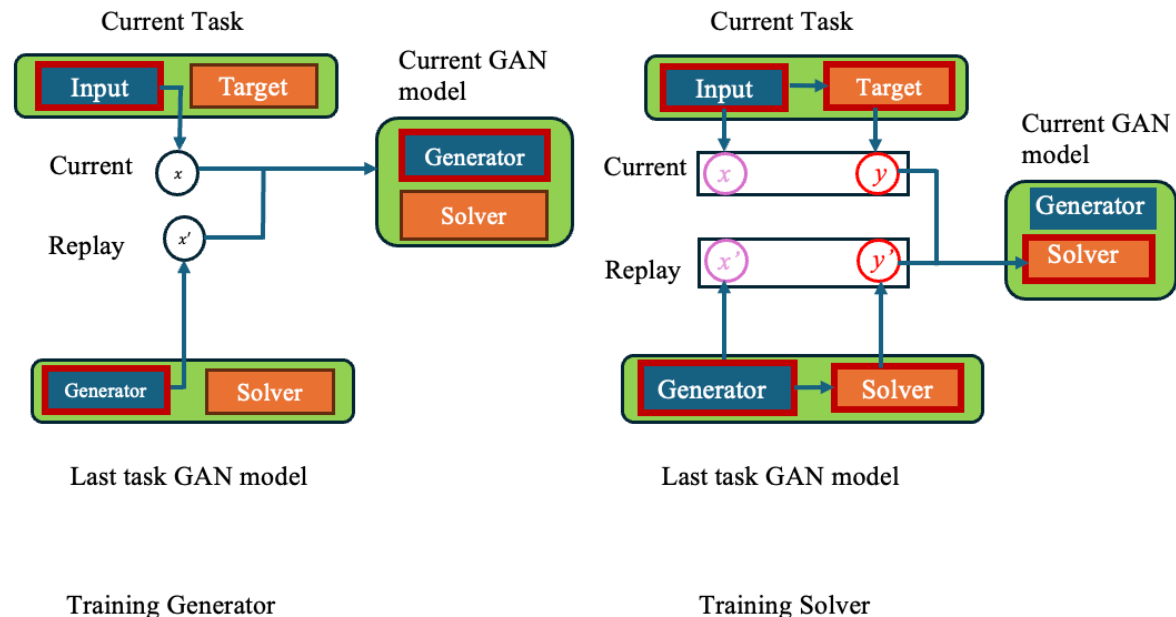


Related Works: Data Based Approach

Exemplar Based Methods: Selecting the most informative exemplars. [1] choose the **most representative ones**. [2] choose the **most discriminative ones**. [3] combines both of methods.

Generative Replays Methods:

[4] use generative model to create synthetic data that resemble previously learned classes.



Related Works: Model Based Approach

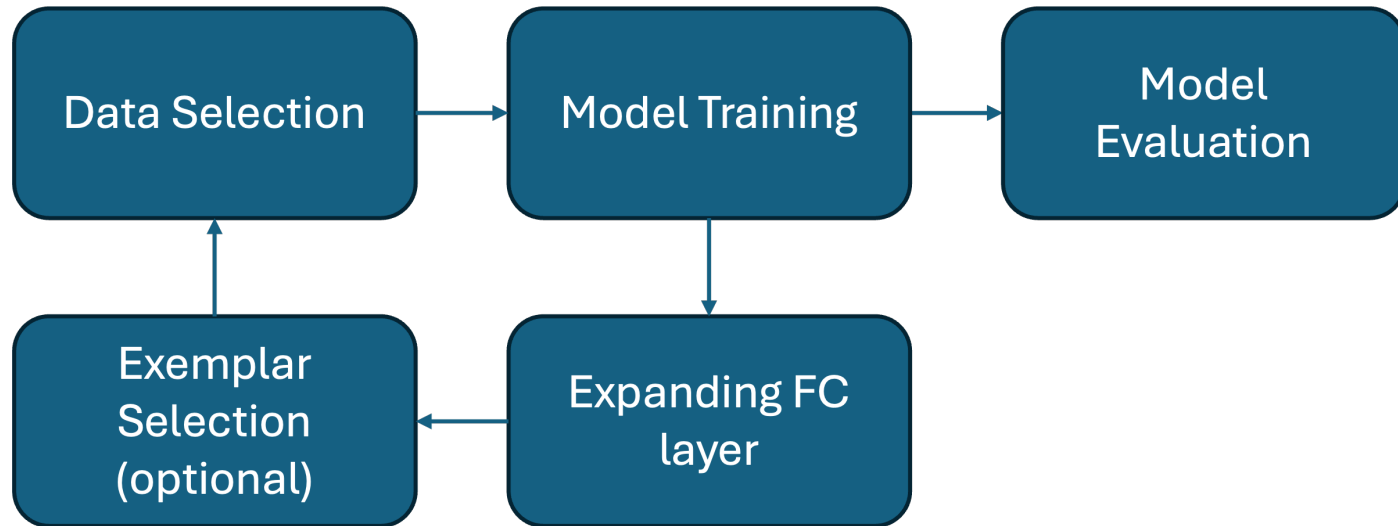
- **Network Expanding Methods:** [5] [6] [7] Adjust the network architecture by adding new component to accommodate new task while preserving previously learned knowledge.
- **Knowledge Distillation Based Methods:** [1] [8] Transfer knowledge of model from former task to current task model.
- **Regularization Based Methods:** [9] Introduce constraints during the training process to prevent the model from significantly altering the weights associated with previously learned classes.



HUST

3. METHODOLOGY

The process is iterative, repeating whenever the model encounters a new task.



Benchmark Methods

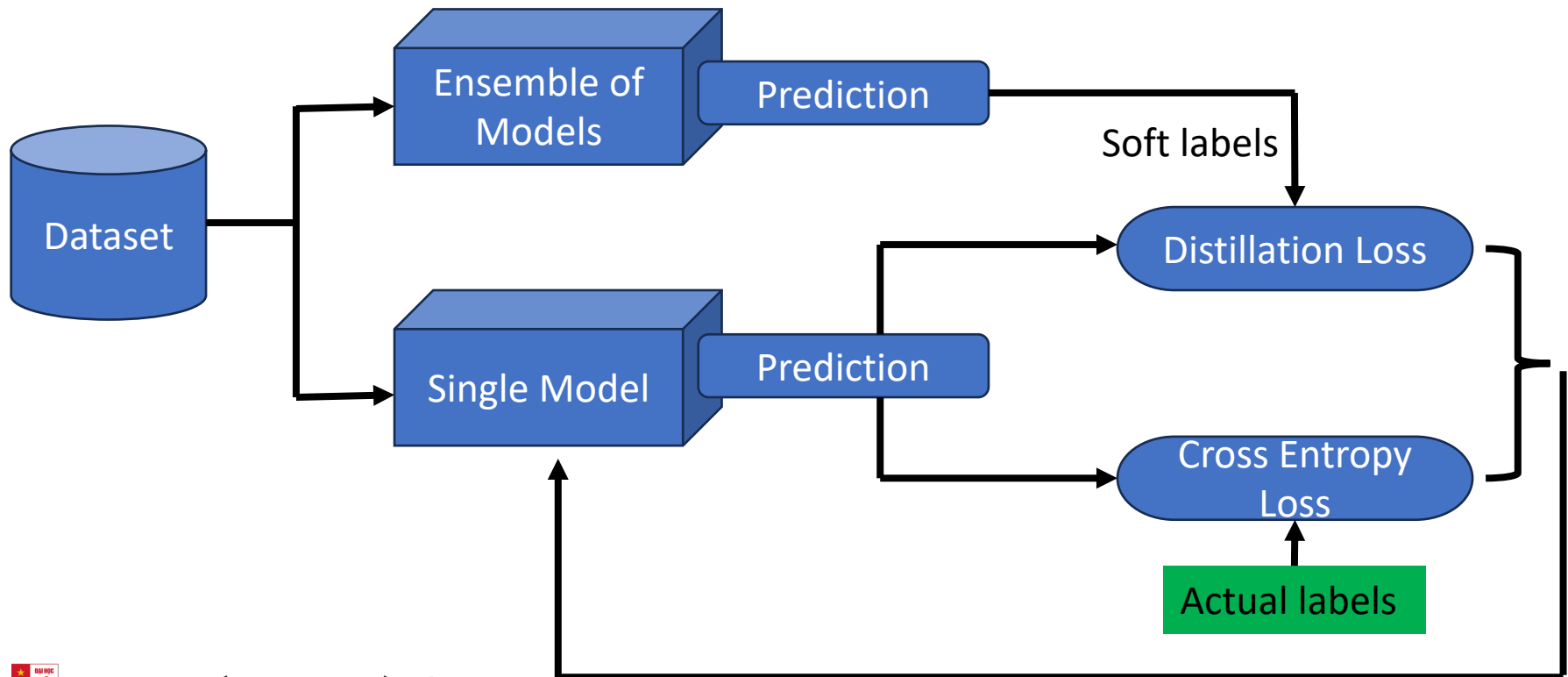
The benchmark methods from 2 approaches:
Knowledge Distillation and Network Expanding.

	Knowledge Distillation	Network Expanding	Exemplar Using
LWF	✓	X	X
ICARL	✓	X	✓
DER	X	Backbone Expand	✓
MEMO	X	Deep layer Expand	✓

LWF: LEARNING WITHOUT FORGETTING

$$\text{Soft label: } \mathcal{S}_{\text{soft}}(z_i) = \frac{\exp(\frac{z_i}{T})}{\sum \exp(\frac{z_j}{T})}$$

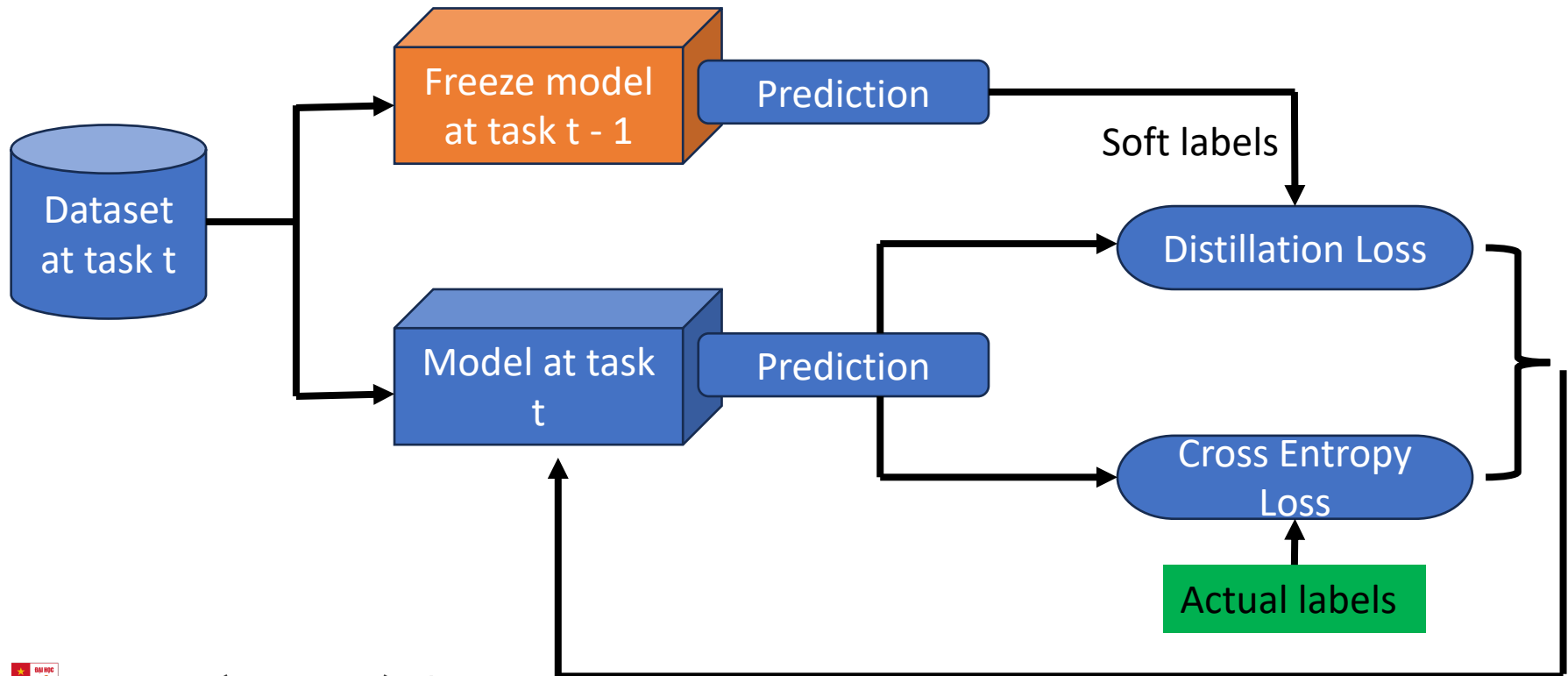
$$\text{Distillation Loss: } \mathcal{L}_{old} = -\mathcal{S}_{\text{soft}}(\mathcal{F}_{\text{teacher}}(\Phi_{\text{teacher}})) * \text{Log}(\mathcal{S}_{\text{soft}}(\mathcal{F}_{\text{student}}(\Phi_{\text{student}})))$$



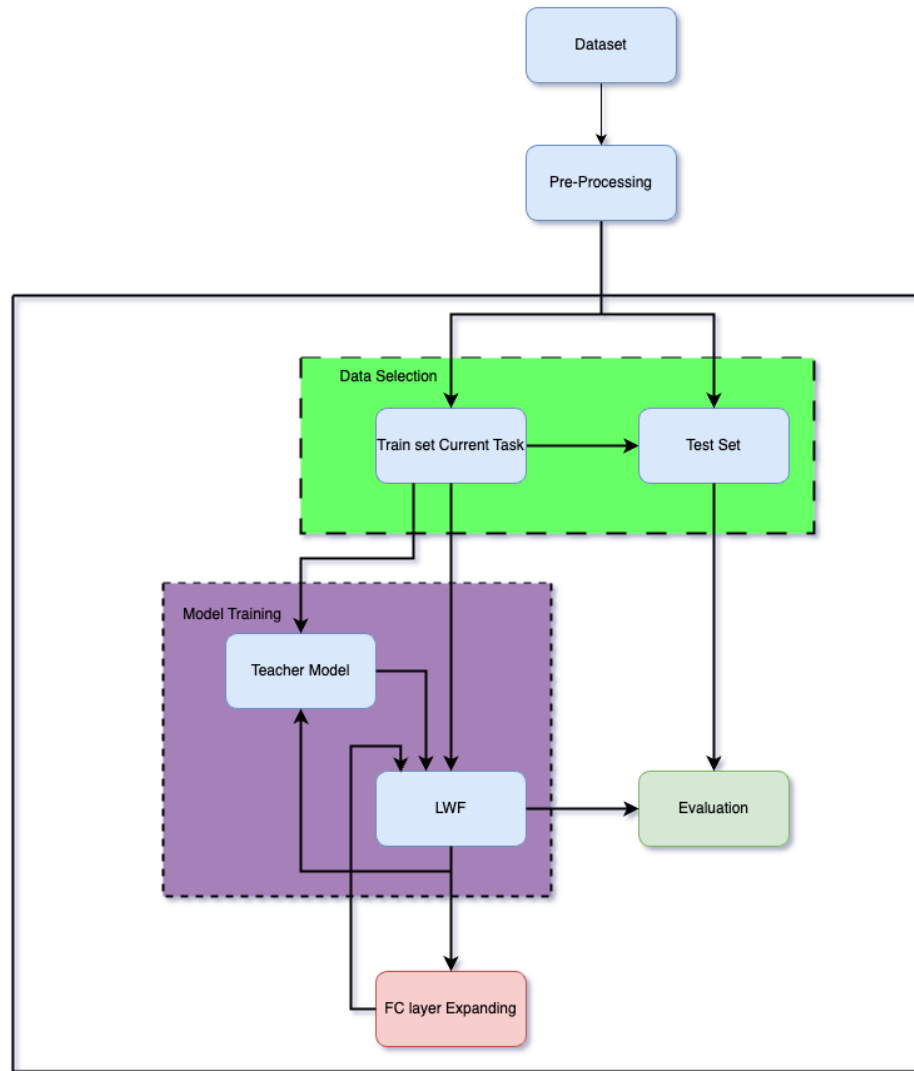
LWF: LEARNING WITHOUT FORGETTING

$$\text{Soft label: } \mathcal{S}_{\text{soft}}(z_i) = \frac{\exp(\frac{z_i}{T})}{\sum \exp(\frac{z_j}{T})}$$

$$\text{Distillation Loss: } \mathcal{L}_{old} = -\mathcal{S}_{soft}(\overline{\mathcal{F}_{t-1}}(\overline{\Phi_{t-1}})) * \text{Log}(\mathcal{S}_{soft}(\mathcal{F}_{t-1}(\Phi_{t-1})))$$



LWF Architecture



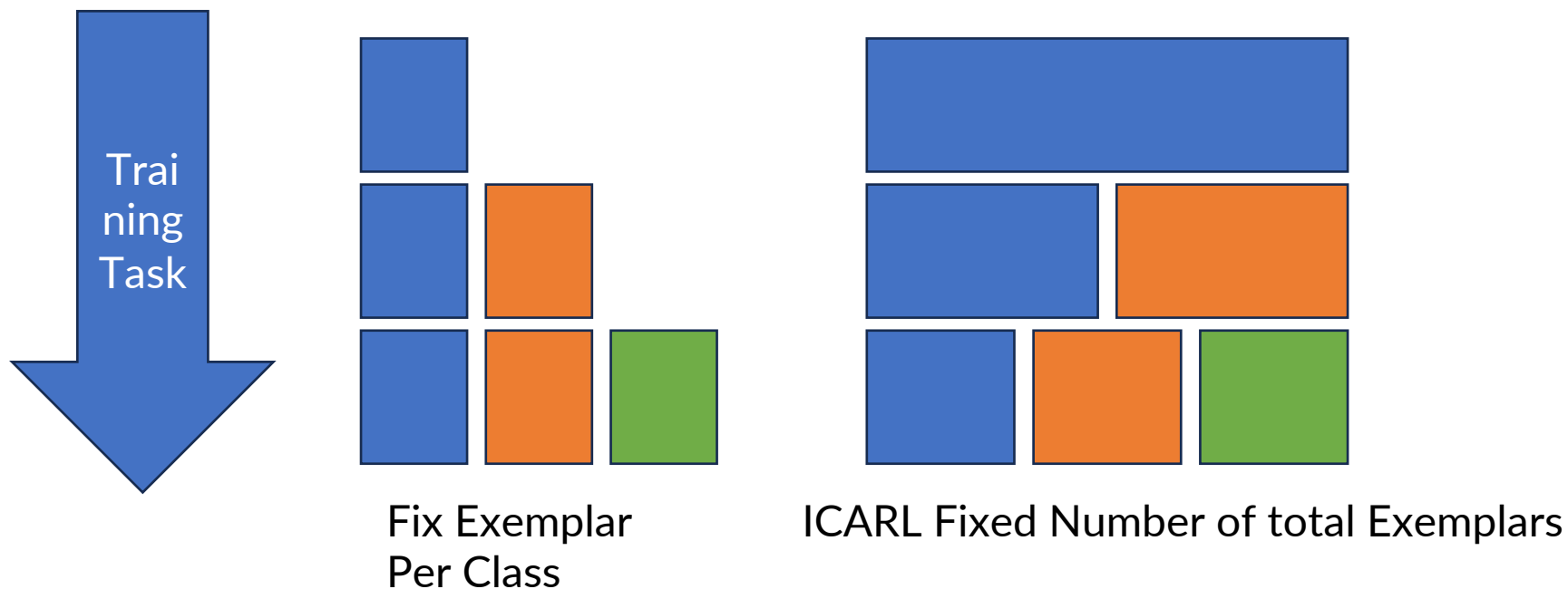
ICARL expands the work of LWF by proposing a combination of three key element:

- Exemplar using.
- Nearest-Mean-of-Exemplar (NME) classifier
- Learning using Knowledge Distillation Loss (same as LWF).

Exemplar Using

ICARL suggests fixing total number of exemplar and employ herding for exemplar selection

Exemplar Management: ICARL proposes storing fixed M number of total exemplars.



ICARL suggests fixing total number of exemplar and employ herding for exemplar selection

Herding Exemplar Selection: Given the instance set $X = [x_1, x_2, \dots, x_n]$ from class y , herding first calculates the class center with current embedding.

$$\mu_y = \frac{1}{|X|} \sum_{x \in X} \Phi(x)$$

Afterward, it calculates and rank the distance of each instance to the class center in ascending order. The exemplars are then selected based on ranking, e.g., the top $[M/y_b]$ instances with the least distance.

Nearest-Mean-of-Exemplar Classifier

Traditionally, ML models predict class labels by choosing the class with the highest probability.

$$y = \operatorname{argmax}(\mathcal{F}(\Phi(x)))$$

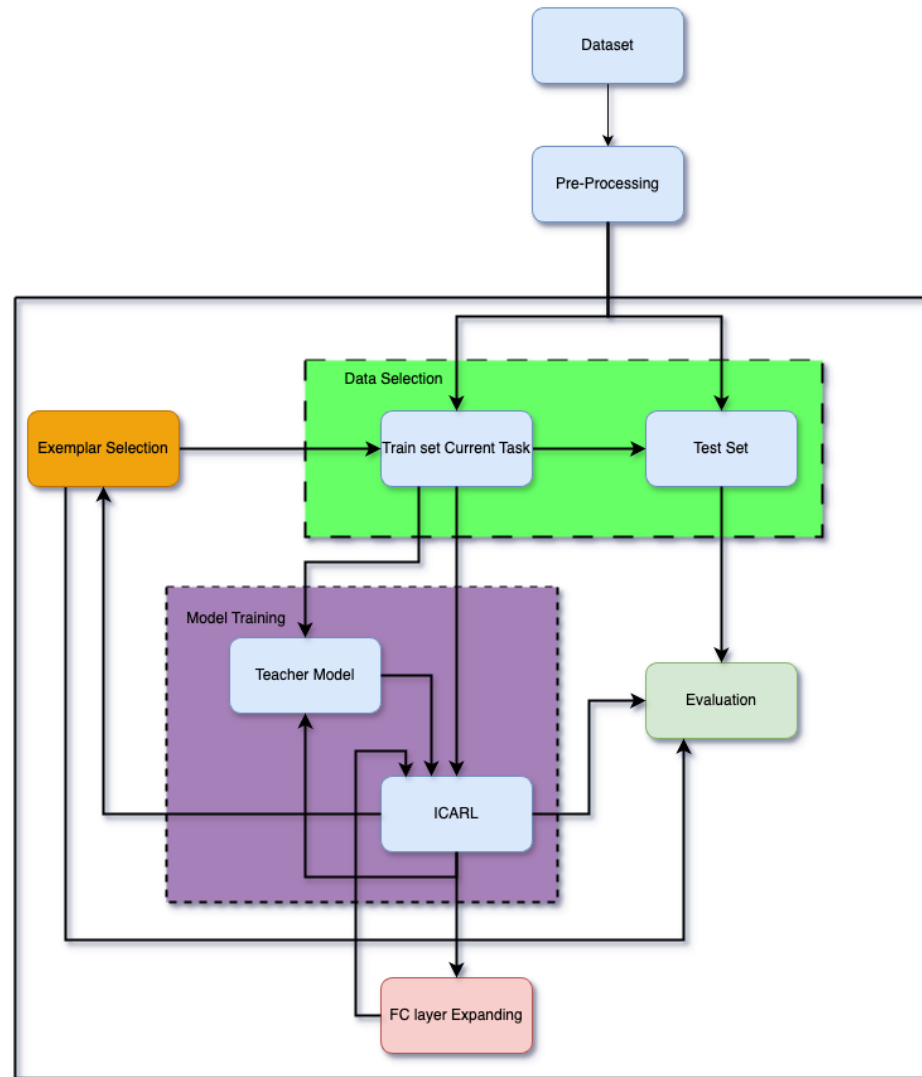
ICARL suggests using NME classifier instead to mitigate catastrophic forgetting. NME first computes a class mean vector for each class encountered so far.

$$\mu_y = \frac{1}{|\epsilon_y|} \sum_{x \in \epsilon_y} \Phi(x)$$

NME computes its feature vector and assigns the class label whose mean vector is closest to the new instance's feature vector:

$$y = \operatorname{argmin}_{y=1,2,3,\dots,t} \|\Phi(x) - \mu_y\|$$

ICARL Architecture

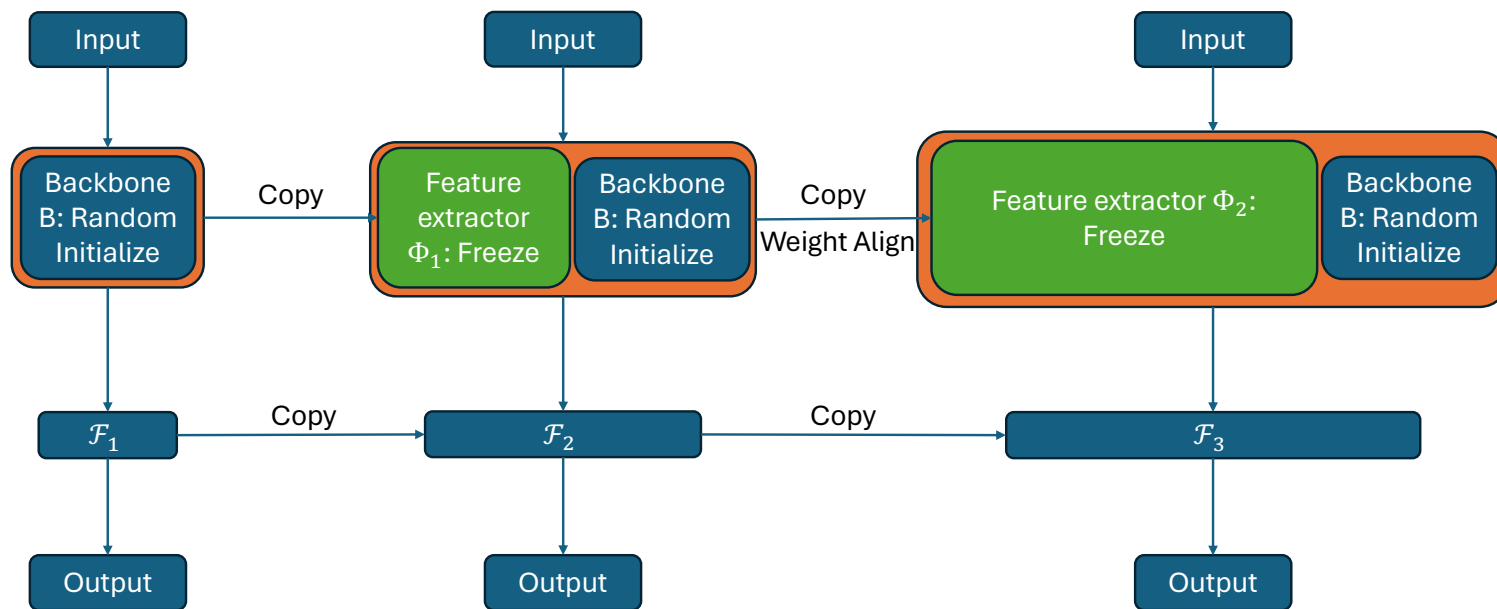


DER: Dynamic Expandable Representation

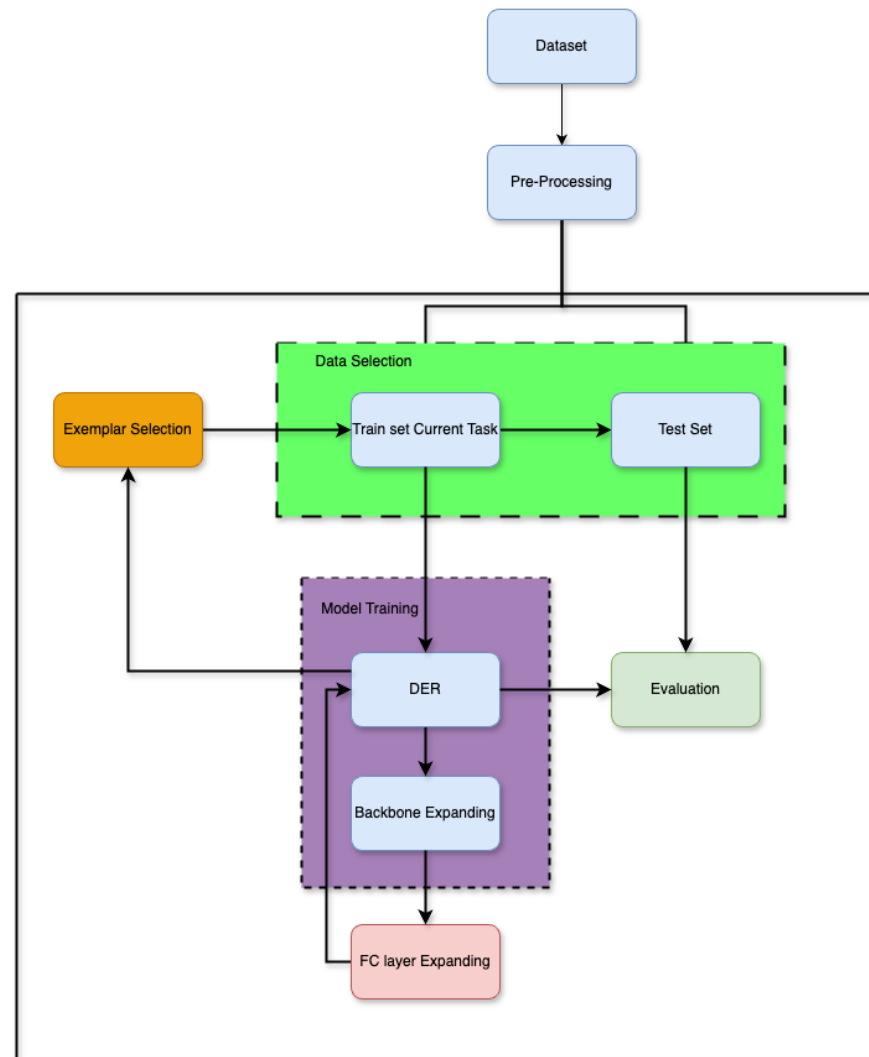
DER expands a new backbone when facing new task.

At step t , the model's feature extractor Φ_t is build by expanding the feature extractor $\overline{\Phi_{t-1}}$ with a newly created backbone $\mathcal{B}$.

$$\Phi_t(x) = [\overline{\Phi_{t-1}}(x), \mathcal{B}(x)]$$

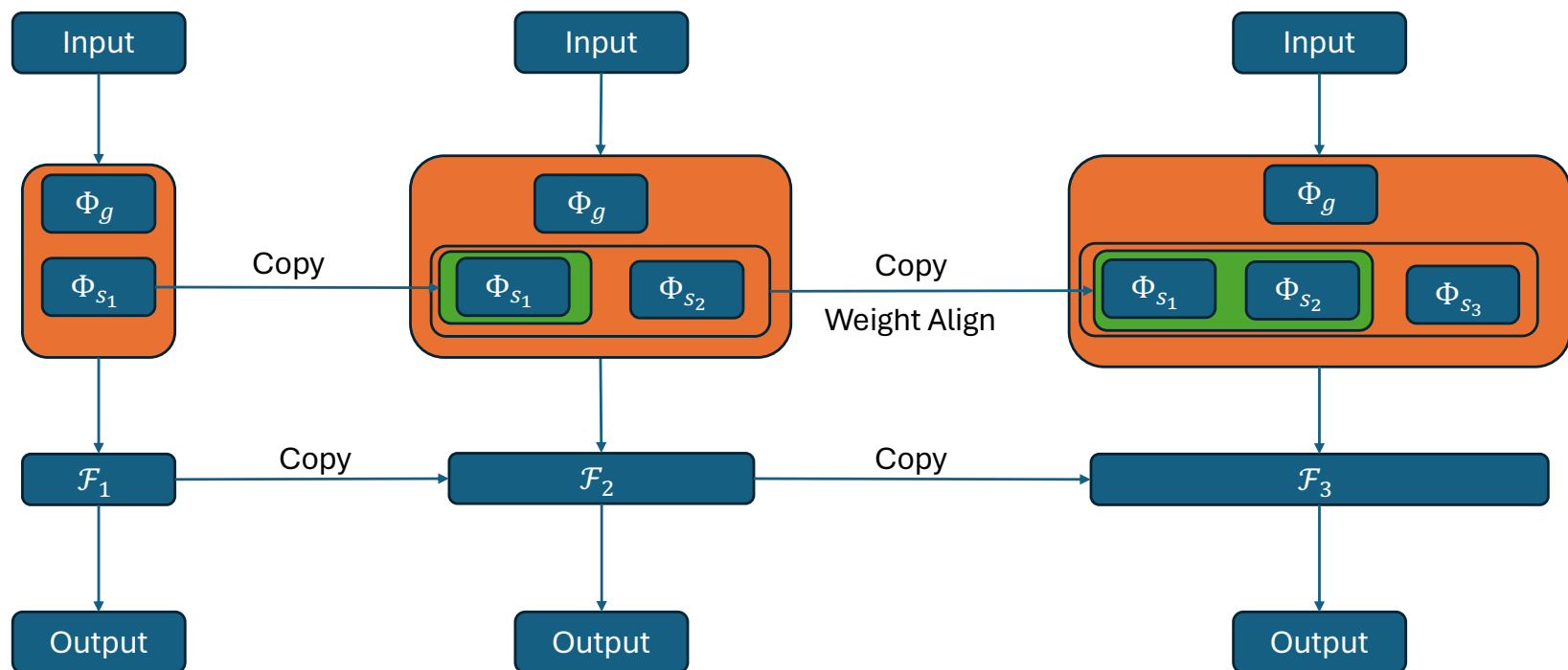


DER Architecture



MEMO: Memory Efficient Expandable Model

Compared to DER, MEMO backbone was decomposing into specialized and generalized layers. MEMO only expand specialized layers.





4. NUMERICAL RESULTS

Numerical results

The methods are tested on 5 datasets: KDD99, CIC-IDS-2017, UNSW-NB15, ToN-IoT-Network and a synthetic dataset.

The methods are evaluated using Last Accuracy and Average Accuracy.

- Last Accuracy: evaluates the model's accuracy on the most recent task it encountered during training.
- Average Accuracy: The accuracy of the model on all tasks.

- Feature encoding
 - One-Hot Encoding
 - Min-Max Encoding
- Feature Processing
 - Duplicate removal
 - Missing data removal
 - Removing features with unique value and id feature.
- Dataset Processing:
 - Removing classes have less than 100 instances in CIC-IDS-2017 dataset and 200 instances in KDD99 dataset.

- Parameter Settings:
 - Epochs: 100
 - Optimizer: Stochastic Gradient Descent
 - Learning rate: 0.1
 - Scheduler: MultistepLR. MultistepLR decays the learning rate by γ once the number of epoch reaches one of the milestones. Milesstones: [25, 50, 75]. γ : 0.1
 - Batchsize: 32
 - Base/Initial classes train: 2
 - Increment classes per task: 2
 - Training order: From class with most instances to least instances

Numerical Results

Evaluation on all datasets

	Finetune	LWF	ICARL	DER	MEMO
KDD99	20,75	68,3	96,59	98,95	95,88
CIC-IDS-2017	22,03	13,15	49,93	32,27	25,96
ToN-IoT-Network	38,26	52,06	66,24	40,25	38,32
UNSW-NB15	23,02	22,46	30,61	27,92	23,13
Synthetic	45,9	50,21	99,99	100	99,88

Table 1: Average Accuracy of all methods on all datasets with exemplar size 2000

- LWF have better accuracy than Finetune in most case.
- Exemplar-based method (ICARL, DER, MEMO) outperforms exemplar-free method.
- ICARL gives competitive results in all datasets.

Numerical Results (2)

Evaluation on different exemplar size: [1000, 500, 250, 100]

Exemplar Size	ICARL	DER	MEMO
1000	99,99	99,96	99,84
500	99,99	100	99,88
250	96,13	99,82	79,84
100	69,25	96,3	64,1

Exemplar Size	ICARL	DER	MEMO
1000	96,39	94,57	94,5
500	80,43	93,02	58,74
250	62,86	51,59	21,02
100	67,11	33,73	20,75

Table 2: Average Accuracy on Synthetic dataset

Table 3: Average Accuracy on KDD99 dataset

- All methods performs equally well with sufficient exemplars (1000 and 500 exemplar on synthetic dataset and 1000 exemplar on KDD99 dataset.
- All methods performances decreased when the size of exemplar decrease.
- DER have better accuracy when exemplar size is large. For really small exemplar size, ICARL performs better.
- DER performs better than MEMO in all exemplar size.

Numerical Results (3)

Evaluation of knowledge-distillation based methods with different temperature hyperparameter $T = [0.25, 0.5, 1, 2, 4, 8]$

T	LWF	ICARL
0,25	32,4	96,04
0,5	41,23	98,53
1	45,9	99,49
2	45,9	96,13
4	45,9	95,59
8	45,9	81,38

Table 4: Average Accuracy on Synthetic dataset 100

T	LWF	ICARL
0,25	20,38	98,42
0,5	20,52	97,03
1	20,75	96,17
2	20,75	96,39
4	20,75	96,17
8	20,75	95,93

Table 5: Average Accuracy on KDD99 dataset 500

- Temperature hyperparameter T have little effect on the performance of LWF.
- Higher T leads to decrease in the performance of ICARL in both datasets.

Numerical Results (4)

Evaluation of DER with NME classifier

Dataset	Normal Classifier	NME Classifier
KDD99 - 1000	94,57	97,88
CIC-IDS-2017 1000	27,38	71,83
ToN-IoT-Network 1000	38,26	95,19
UNSW-NB15 1000	24,32	31,37
Synthetic 100	96,3	99,77

Table 6: Average Accuracy of DER with NME on all dataset, compared to normal DER

DER model with NME Classifier performs better than traditional DER in all datasets.



HUST

5. CONCLUSION

Conclusion

- All benchmark methods perform better than the traditional model in incremental learning setup. However, their effectiveness heavily relies on the size of the exemplar set.
- Soft label target is not effective in knowledge distillation approach.
- NME classifier performs well in CIL compared to traditional classifier.
- DER, combined with the NME classifier, achieve highest accuracy. However, DER is more resource expensive than other.

A large, stylized graphic of the HUST logo, composed of concentric circles of dots in a lighter shade of red, set against a solid red background.

HUST

THANK YOU !

References

- [1] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “Icarl: Incremental classifier and representation learning,”
- [2] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, “Riemannian walk for incremental learning: Understanding forgetting and intransigence,”
- [3] J. Bang, H. Kim, Y. Yoo, J.-W. Ha, and J. Choi, “Rainbow memory: Continual learning with a memory of diverse samples,”
- [4] H. Shin, J. K. Lee, J. Kim, and J. Kim, “Continual learning with deep generative replay,”
- [5] S. Yan, J. Xie, and X. He, “Der: Dynamically expandable representation for class incremental learning,”

References (2)

- [6] F.-Y. Wang, D.-W. Zhou, H.-J. Ye, and D.-C. Zhan, “Foster: Feature boosting and compression for class-incremental learning,”
- [7] D.-W. Zhou, Q.-W. Wang, H.-J. Ye, and D.-C. Zhan, “A model or 603 exemplars: Towards memory-efficient class-incremental learning,”
- [8] Z. Li and D. Hoiem, “Learning without forgetting,”
- [9] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al., “Overcoming catastrophic forgetting in neural networks,”

The logo for HUST (Ho Chi Minh University of Science) is displayed in white, bold, sans-serif capital letters. It is centered within a circular graphic composed of numerous small dots of varying opacity, creating a sense of depth and movement. The entire graphic is set against a solid red background.

HUST

APPENDIX

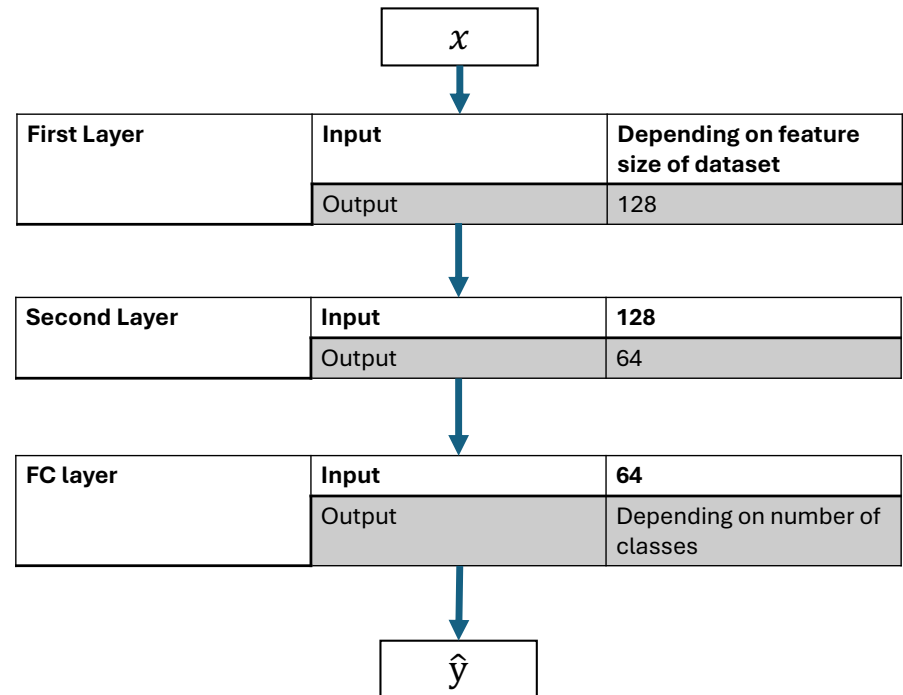
Backbone Architecture

LWF: Store 2 backbones at the time, one for the model at the current task, one for model from previous task

ICARL: Store 2 backbones at the time, one for the model at the current task, one for model from previous task

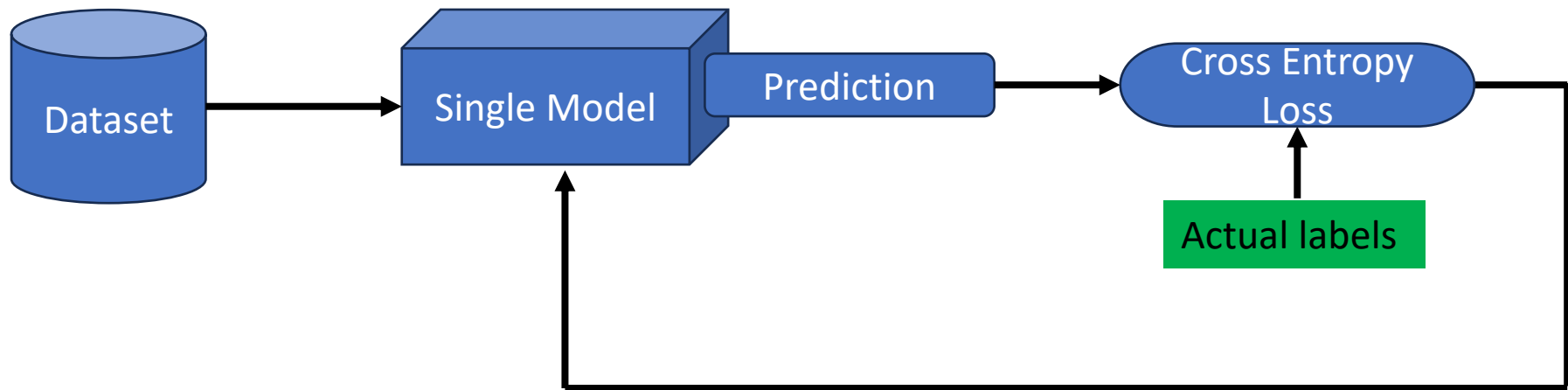
DER: Adding a backbone whenever facing new task

MEMO: Adding second layer whenever facing new task



Backbone Architecture

LWF: LEARNING WITHOUT FORGETTING



Idea: An ensemble of models predict better than a single model.

Knowledge distillation: Boost the accuracy of single model by transfer knowledge from an ensemble of models to the single model.